

Report
On
*SwiftVisa: AI-Based Visa Eligibility
Screening Agent*



Virtual Internship 6.0 – Batch: 6

Mentor: Mr. Siddarth

Submitted By:

Ms. Shambhavi Raj

(Vishwakarma Institute of Information Technology, Pune)

I. Executive Summary

"SwiftVisa.ai" is an intelligent RAG tool-a "Retrieval-Augmented Generation" tool that democratizes access to complex immigration knowledge. The AI tool works like a copilot for those planning to immigrate to the USA, UK, and Canada, guiding them through complex related regulations concerning visa types. Thanks to the extremely fast inference provided by "Groq" and the "Llama-3.3" model, equipped with reason capabilities, this AI tool gives very reliable and source-driven advice concerning documents, financial aspects, and processing times.

II. System Architecture: RAG Pipeline

It is supposed to be a modular pipeline of 5 steps and gives a highly accurate result in low latency.

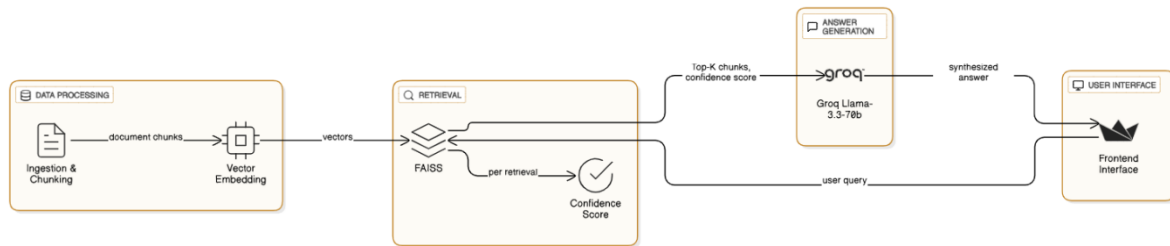


Fig. 1 Architecture Diagram

Step 1: Consumption and Chunking

- **Objective:** To process official PDF regulatory documents.
- **Implementation:** For this, the raw PDF files, such as USA Visa.pdf, available in the countries/ directory are used as inputs. Here, it uses some sort of chunking algorithm that could have potentially been used to divide large pieces of lawful text, like the ones used here, into smaller meaningful pieces called Document Chunks.
- **Pseudo Code:** step1_extract.py

```
ALGORITHM ExtractTextFromPDFs(pdf_path_list):  
    INITIALIZE empty list 'chunks'
```

```
    PRINT "Starting Step 1: Extraction..."
```

```
    FOR EACH file_path IN pdf_path_list:  
        IF file_path DOES NOT EXIST:  
            SKIP to next file
```

```
        # Parse PDF  
        LOAD PDF using PdfReader(file_path)  
        GET filename FROM file_path
```

```

# Iterate through pages
FOR EACH page_number, page IN ENUMERATE (reader.pages):
    text = EXTRACT_TEXT (page)

    # Create a clean data object
    IF text IS NOT EMPTY:
        chunk_object = {
            "id": GENERATE_UUID (),
            "text": TRIM (text),
            "source": filename + " (Page " + page_number + ")"
        }
        APPEND chunk_object TO chunks

# Save Data
OPEN "chunks.jsonl" FOR WRITING:
    FOR EACH chunk IN chunks:
        WRITE JSON (chunk) TO file

PRINT "Saved " + LENGTH (chunks) + " chunks."
RETURN chunks

```

Step 2: Vector Embedding

- **Objective:** The text can be represented by vectors in a machine-readable form.
- **Implementation:** The text input is processed using the sentence-transformers library that has the capability of embedding text, thereby embedding the text into a higher dimensional vector space that enables this system to perform semantic searches that can recognize the intent behind the input search query rather than keyword searches.
- **Pseudo Code:** step2_embed.py

```

ALGORITHM CreateEmbeddings(chunks_list):
    PRINT "Starting Step 2: Embedding..."

    # 1. Initialize Model
    # Using 'all-MiniLM-L6-v2' for efficient local embedding
    LOAD Model SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

    # 2. Prepare Text
    EXTRACT list_of_texts FROM chunks_list

    # 3. Generate Vectors
    # convert_to_numpy=True ensures compatibility with FAISS
    vectors = Model.ENCODER(list_of_texts, convert_to_numpy=True)

    # 4. Normalize (Crucial for Cosine Similarity)
    # L2 Normalization makes vector length = 1
    # This allows Dot Product to equal Cosine Similarity
    CALL FAISS.Normalize_L2(vectors)

```

RETURN vectors, Model

- Pseudo Code: step3_index.py

ALGORITHM CreateFAISSIndex(vectors):

PRINT "Starting Step 3: Indexing..."

1. Determine Dimensions

e.g., MiniLM usually produces 384 dimensions

dimension_size = vectors.SHAPE[1]

2. Create Index

IndexFlatIP = Exact Search using Inner Product (Cosine Sim)

index = INITIALIZE FAISS.IndexFlatIP(dimension_size)

3. Add Vectors

index.ADD(vectors)

4. Persist to Disk

SAVE_INDEX(index, "faiss_index.bin")

PRINT "Index saved with " + index.TOTAL_VECTORS + " entries."

RETURN index

Step 3: Semantic Retrieval (FAISS)

- **Result:** Extract the most pertinent information milliseconds after conducting a search.
- **Implementation:** The implementation details involve the usage of a Vector Database named FAISS (Facebook AI Similarity Search). Upon receiving a query from a user, it determines the Cosine Similarity for the query vector with respect to document vectors, acquiring the "Top K" most significant chunks.
- **Key Feature Description:** Every retrieval has a corresponding Confidence Score that measures how relevant the obtained information is.
- **Pseudo Code:** step4_retrieval.py

ALGORITHM SearchIndex(user_query, index, chunks, model, k=3):

1. Convert query to vector

query_vector = model.ENCODE(user_query)

2. Normalize Vector (Critical for Cosine Similarity)

NORMALIZE query_vector (L2 Normalization)

3. FAISS Search

Returns distances (scores) and indices of nearest k neighbors

distances, indices = index.SEARCH(query_vector, k)

```

INITIALIZE results list
SET top_confidence_score = 0.0

# Extract Confidence
IF distances has results:
    top_confidence_score = First Distance Value (Best Match)

# Retrieve Text Chunks
FOR EACH idx IN indices:
    IF idx is valid:
        GET chunk at chunks[idx]
        APPEND chunk TO results

RETURN results, top_confidence_score

```

Step 4: LLM Augmentation (Groq)

- **Engine:** It uses the Groq API running the Llama-3.3-70b-vers. The recovered pieces of text are then combined with the question asked by the user to create a strict system prompt. Groq analyses this system prompt and provides an answer based entirely on this context. This has served to reduce several hallucinations.
- **Performance:** Groq's LPUs allow for virtually instantaneous inference that solves the "Time to First Token" problem prevalent in RAG applications.
- Pseudo Code: step5_generate.py

```

ALGORITHM GenerateAnswerStream(query, context_chunks):
    # 1. Prepare Context Block
    context_text = COMBINE (chunk.source + "\n" + chunk.text FOR chunk IN
context_chunks)

    # 2. Define System Persona (Strict Guardrails)
    system_prompt = "You are SwiftVisa. Answer using ONLY the provided context.
        If answer is missing, state 'I don't have that information'.
        Be professional."

    # 3. Call Groq API with Streaming
    TRY:
        stream_response = CALL GroqAPI.ChatCompletion(
            model = "llama-3.3-70b-versatile",
            messages = [
                {role: "system", content: system_prompt},
                {role: "user", content: "Context: " + context_text + "\nQuestion: " + query}
            ],
            stream = True # Enable Token Streaming
        )

    # 4. Yield Tokens Real-time

```

```

FOR EACH chunk IN stream_response:
    IF chunk has content:
        YIELD content
CATCH Error:
    YIELD "System Error: " + Error Message

```

- **Pseudo Code:** main.py

```

ALGORITHM RunSwiftVisaPipeline:
    DEFINE base_directory AS "Milestone_02_Shambhavi"
    DEFINE file_paths FOR chunks, index, logs, and PDFs

    FUNCTION LogInteraction(query, answer, score):
        CREATE log_entry WITH:
            id = generate_unique_id()
            timestamp = current_time()
            query = user_query
            answer = system_answer
            confidence = confidence_score
        APPEND log_entry TO log_file
        SAVE log_file

    FUNCTION Main():
        INITIALIZE system_variables (chunks, index, model)

        # Logic Switch: Load Existing vs Create New
        IF chunks_file EXISTS AND index_file EXISTS:
            PRINT "Loading existing data..."
            chunks = LOAD from chunks_file
            index = LOAD from index_file
            model = LOAD embedding_model
        ELSE:
            PRINT "Initializing new data pipeline..."
            # Step 1: Extraction
            chunks = CALL ExtractTextFromPDFs(pdf_files)
            SAVE chunks TO chunks_file

            # Step 2: Embedding
            vectors, model = CALL CreateEmbeddings(chunks)

            # Step 3: Indexing
            index = CALL CreateFAISSIndex(vectors)
            SAVE index TO index_file

        PRINT "System Ready"

        # Interactive Chat Loop
        WHILE True:
            query = GET user_input
            IF query IS "quit": BREAK

```

```

# Step 4: Retrieval
relevant_docs, score = CALL RetrieveDocuments(query, index, chunks)

# Step 5: Generation
PRINT "Groq is thinking..."
answer = CALL GenerateAnswer(query, relevant_docs)

PRINT answer AND score
CALL LogInteraction(query, answer, score)
EXECUTE Main()

```

Step 5: Creating the Frontend Interface (Streamlit)

- **Result:** This document will then be transformed into an interactive user interface.
- **Implementation:** The Streamlit interface has been designed with a professional looking "glass morphism" theme, and the design includes the functionalities for session handling, conversation log, and real time text streaming.
- **Pseudo Code:** app.py

ALGORITHM SwiftVisaFrontend:

```

INITIALIZE Streamlit Page Configuration
APPLY Custom CSS (Theme, Chat Bubbles, Sidebar, Fonts)

# State Management Initialization
IF session_state NOT Initialized:
    LOAD RAG System (Chunks, Index, Model)
    LOAD Conversation History from JSON
    SET Initialized = True

FUNCTION CreateNewChat():
    id = generate_unique_id()
    CREATE new_conversation_object(id, title="New Chat", messages=[])
    SET current_chat_id = id
    SAVE history

FUNCTION SubmitQuery():
    GET user_input
    APPEND user_input TO current_chat messages
    SET processing_flag = True

# --- UI RENDERING ---

# 1. Sidebar Render
DISPLAY Sidebar Brand Header
BUTTON "New Chat" -> TRIGGER CreateNewChat()

DISPLAY Recent Chats List:

```

```

FOR EACH chat IN history (Sorted by Date):
    DISPLAY Chat Title Button (Click to Switch Chat)
    DISPLAY Edit/Delete Options

# 2. Main Content Render
DISPLAY Main Header "SwiftVisa.ai"

IF current_chat_id IS set:
    # Render Chat History
    FOR EACH message IN current_chat_messages:
        IF role IS 'user':
            DISPLAY User Bubble (Right Aligned)
        ELSE:
            DISPLAY Assistant Bubble (Left Aligned) WITH Confidence Badge

# Handle New Query Processing (RAG)
IF processing_flag IS True:
    query = GetLastUserMessage()

    # Context Assembly
    history_context = FormatPreviousMessages(chat_history)
    full_prompt = history_context + query

    # RAG Retrieval
    relevant_docs, confidence = CALL RetrieveDocuments(full_prompt)

    # Streaming Generation (Real-time UI Update)
    CREATE Placeholder
    full_response = ""

    FOR chunk IN CALL GenerateAnswerStream(full_prompt, relevant_docs):
        APPEND chunk TO full_response
        UPDATE Placeholder WITH full_response

    # Save Interaction
    APPEND {role: 'assistant', content: full_response, confidence: confidence} TO
chat_messages
    SAVE history
    SET processing_flag = False

ELSE:
    DISPLAY Welcome Screen (Empty State / Quick Prompts)

# 3. Input Area
DISPLAY Text Input Field (Bottom Fixed)
ON_SUBMIT -> CALL SubmitQuery()

```


III. *User Interface and Key Features*

The UI was gradually improved to produce a “production grade” interface like what you would see in current AI models such as Gemini or ChatGPT.

A. *Smart Dashboard*

- i. **Landing Page:** Hero Section is centered, with tagline "Your AI Copilot for Seamless Global Travel."
- ii. **Quick Actions:** These are interactive cards that activate pre-defined prompt tasks like “Canada Proof of Funds” or “US Student Visa Checklist.”

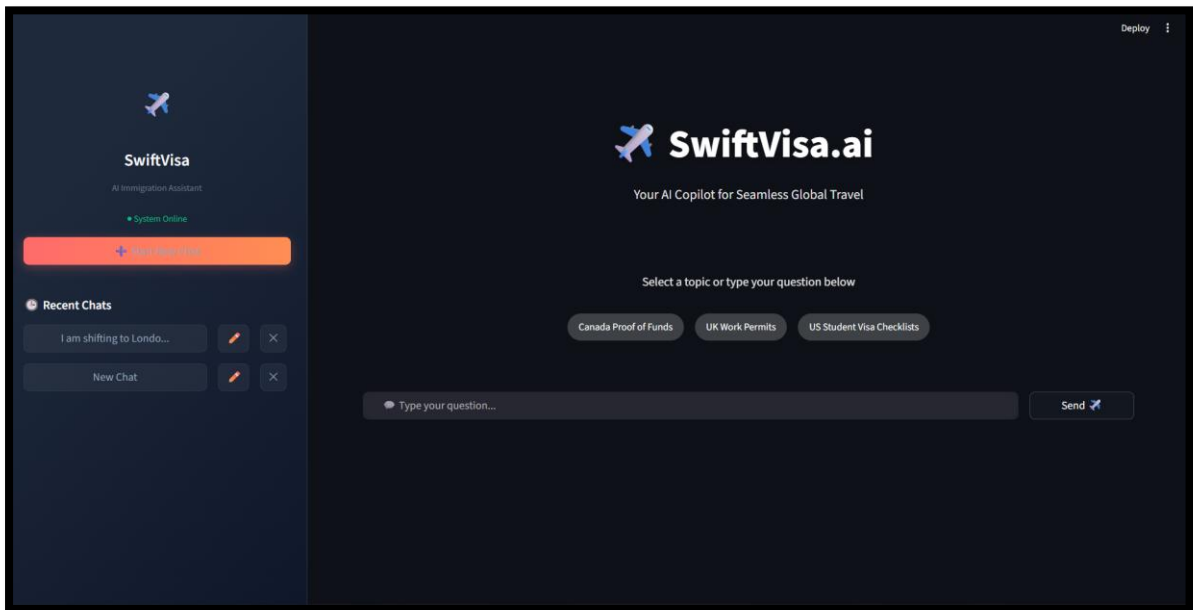


Fig. 2 Homepage

B. *Advanced Chat Interface*

- i. **Real-Time Streaming:** The answers are fed back in a stream of one token at a time, providing immediate interactive responses.
- ii. **Visual Distinction:** Messages from other people are displayed in Blue bubbles (right-aligned), whereas responses from Artificial Intelligence are displayed in simple and white designs (left-aligned).
- iii. **Continuous Chat:** There is no shortage of context in the system, and users can ask vague follow-up questions like “How much does it cost?” or “What about for students?” and get the correct context from the previous conversations.

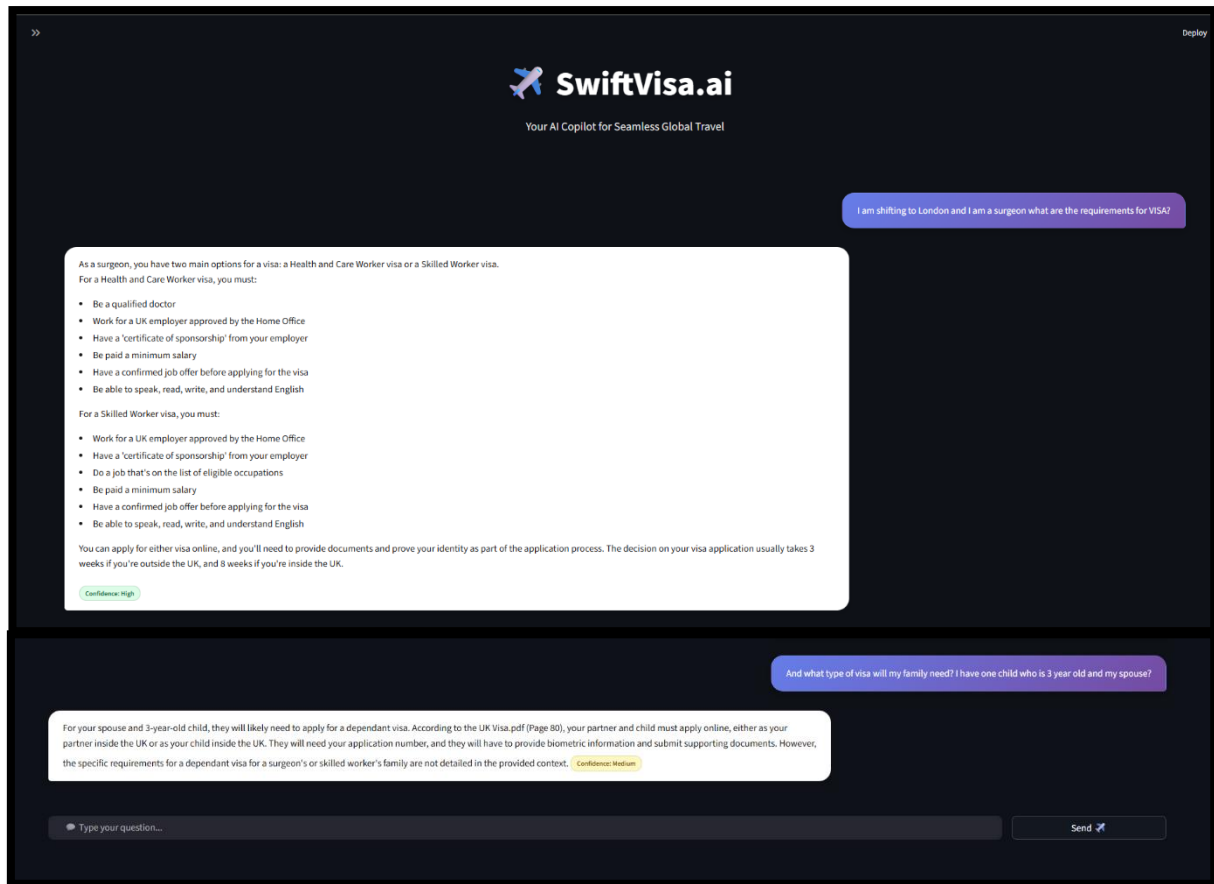


Fig. 3 Conversation Page

C. Robust History Management

- i. **Time-Grouped History:** Chat sessions can be segmented based upon recency.
- ii. **Popover Menu:** A menu was implemented for each chat message. This is achieved using Streamlit's popover feature to include "Rename" and "Delete" features on the dashboard without cluttering an already congested design.
- iii. **Navigation:** Active "SwiftVisa" button that takes one directly to the homepage.

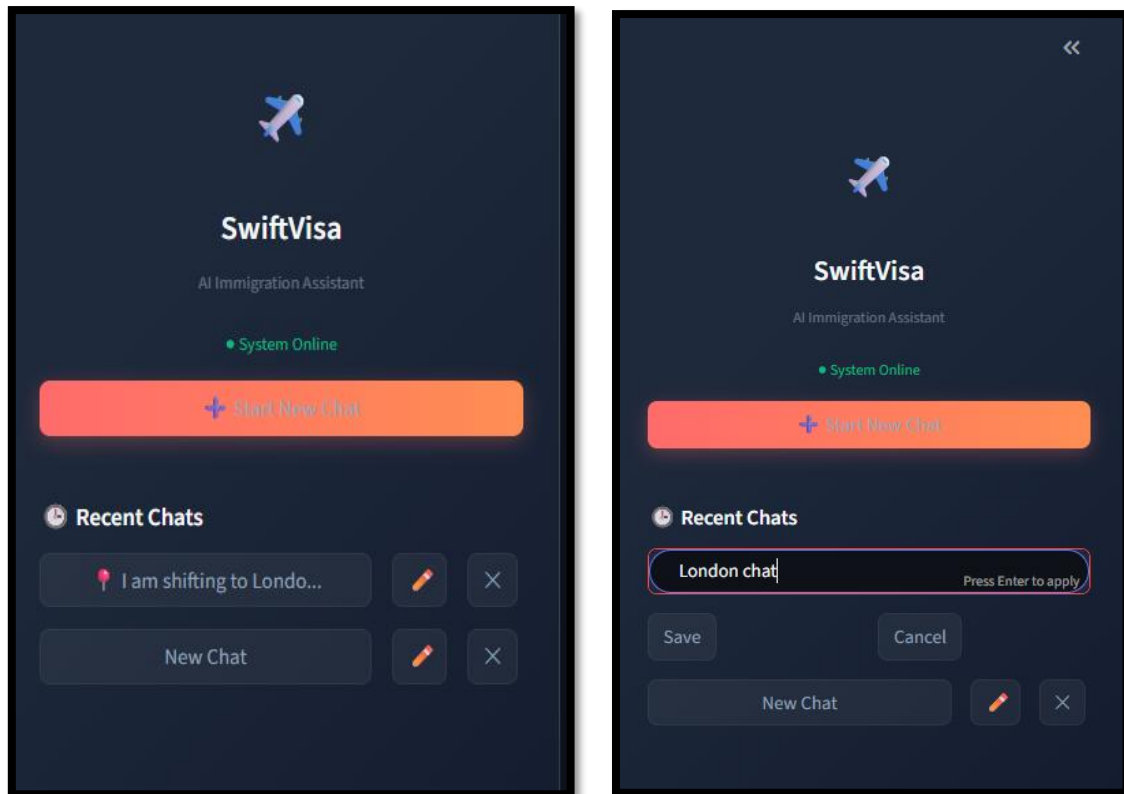


Fig. 4 Sidebar Menu

IV. *Technical Highlights*

- i. **Universal Visibility Fix:** A CSS reset file was created, which enforced High Contrast Mode (Black and White), and this ensured that the app was viewable on all systems, irrespective of the device theme.
- ii. **Smart Path Handling:** The backend has smart path handling to detect, by logic coded into it, whether it is running in the root directory or subdirectory. This prevents any cases of `FileNotFoundException`.
- iii. **Confidence Scoring:** It allows the FAISS similarity score to be presented to the user, with transparency about how well the AI matched the question to the document.

V. *Conclusion*

SwiftVisa.ai closes the distance between immigration complexity and technology usability perfectly. By using Groq's speed and the precision offered by FAISS, it provides a service that is not only conversational but also informative and transparent. The clean interface allows users to properly manage their consultation sessions and makes it a complete service for all international travelers.